EWB AWS VPC Peering Student Project

Connect two VPCs. Prove private connectivity.

Student	Santanu Mondal
Project	EWB AWS VPC Peering Student Project
AWS Region	us-west-2 (Oregon)
Execution date	October 23, 2026
Environment	AWS Skill Builder lab sandbox (AWSLabsUser role)
Account / role	AWSLabsUser (managed lab sandbox)

1. Executive Summary

1.1 Objective

This project builds the network foundation for private connectivity between two isolated Amazon VPCs using a **VPC Peering Connection**, and defines a verification plan that proves: (a) workloads in separate VPCs communicate over private IP addresses without crossing the public internet; (b) administrative access to private instances is possible only through a hardened bastion host; and (c) reachability depends on routing in *both* VPCs plus correct security group rules, not on the peering state alone.

1.2 Architecture overview

- **VPC A (ewb-vpc-a)** contains a public subnet (planned bastion host, internet-facing through an Internet Gateway) and a private subnet (planned App-A).
- **VPC B (ewb-vpc-b)** contains a single private subnet (planned App-B) with no Internet Gateway and no public addressing.
- A **peering connection** links the two VPCs. Static routes in each VPC direct traffic for the remote CIDR through the peering connection.

1.3 Non-overlapping CIDR plan and IP design rationale

Network	CIDR	Role	Usable IPs*
VPC A	10.10.0.0/16	Bastion + App-A	65,531
ewb-a-public	10.10.1.0/24	Bastion host	251
ewb-a-private	10.10.2.0/24	App-A	251
VPC B	10.20.0.0/16	App-B	65,531
ewb-b-private	10.20.1.0/24	App-B	251

*AWS reserves 5 addresses per subnet (network address, VPC router, DNS, reserved for future use, broadcast).

- **Non-overlapping ranges are mandatory.** AWS refuses to peer VPCs with overlapping CIDRs, and routing would be ambiguous. 10.10.0.0/16 and 10.20.0.0/16 share no addresses.

- **RFC 1918 private space** keeps all traffic non-routable on the public internet.
- **Readable octet scheme:** the second octet identifies the VPC (10 = A, 20 = B) and the third identifies the tier (1 = public, 2 = private in A; 1 = private in B), making route tables and security rules easy to audit.
- **/16 VPCs with /24 subnets** leave room for more tiers or Availability Zones without renumbering.

2. Resource Inventory and Evidence

2.1 VPCs, subnets, gateways and peering

Resource	Name	ID	CIDR / Detail
VPC	ewb-vpc-a	vpc-00595e2a2cfb858dd	10.10.0.0/16; State: Available; DNS resolution enabled
VPC	ewb-vpc-b	vpc-0d558ceae7d6a326c	10.20.0.0/16
Subnet	ewb-a-public	subnet-0fee352033ec319c6	10.10.1.0/24
Subnet	ewb-a-private	subnet-0a000387dc83a4d12	10.10.2.0/24
Subnet	ewb-b-private	subnet-05debd941ed0e600a	10.20.1.0/24
Internet Gateway	ewb-igw-a	igw-0a44f93c2902ce0ee	Attached to ewb-vpc-a
Peering Connection	ewb-peering-a-to-b	pcx-0a53069a0b0424408	Requester: ewb-vpc-a (10.10.0.0/16); Accepter: ewb-vpc-b; **Status: Pending acceptance** (expires Oct 29, 2026)

2.2 Route tables

Route table	ID	Associated subnet	Routes as configured
ewb-rt-a-public	rtb-0b9a24627a866393f	ewb-a-public	10.10.0.0/16 -> local (Active) 0.0.0.0/0 -> igw-0a44f93c2902ce0ee (Active) *Peering route for 10.20.0.0/16 required, see Section 3.3*
ewb-rt-a-private	rtb-0a1e8f41cdd77a5d6	ewb-a-private	10.10.0.0/16 -> local (Active) 10.20.0.0/16 -> pcx-0a53069a0b0424408 (**Blackhole**)
ewb-rt-b-private	rtb-026be207ac9b3a54c	ewb-b-private	10.20.0.0/16 -> local (Active) 10.10.0.0/16 -> pcx-0a53069a0b0424408 (**Blackhole**)
Main (VPC A)	rtb-066ad9197c76fd172	None explicit	10.10.0.0/16 -> local

Blackhole is the expected status of a peering route while the connection has not been accepted. It changes to Active once the accepter side accepts the request.

2.3 Security groups

Security group	ID	VPC	Inbound rules
ewb-sg-bastion	sg-0cb096ef048164789	VPC A	TCP 22 from Admin public IP /32
ewb-sg-app-a	(record from console)	VPC A	TCP 22 from 10.10.1.10/32 (bastion) ICMP Echo Request from 10.20.1.10/32 (App-B)
ewb-sg-app-b	sg-02f97011ae2d414a4	VPC B	TCP 22 from 10.10.1.10/32 (bastion) ICMP Echo Request from 10.10.2.10/32 (App-A)

2.4 Planned compute (not launched, see Section 5)

Instance	Subnet	Private IP	Security group	Public IP
Bastion	ewb-a-public	10.10.1.10	ewb-sg-bastion	Yes
App-A	ewb-a-private	10.10.2.10	ewb-sg-app-a	No
App-B	ewb-b-private	10.20.1.10	ewb-sg-app-b	No

2.5 Screenshot evidence

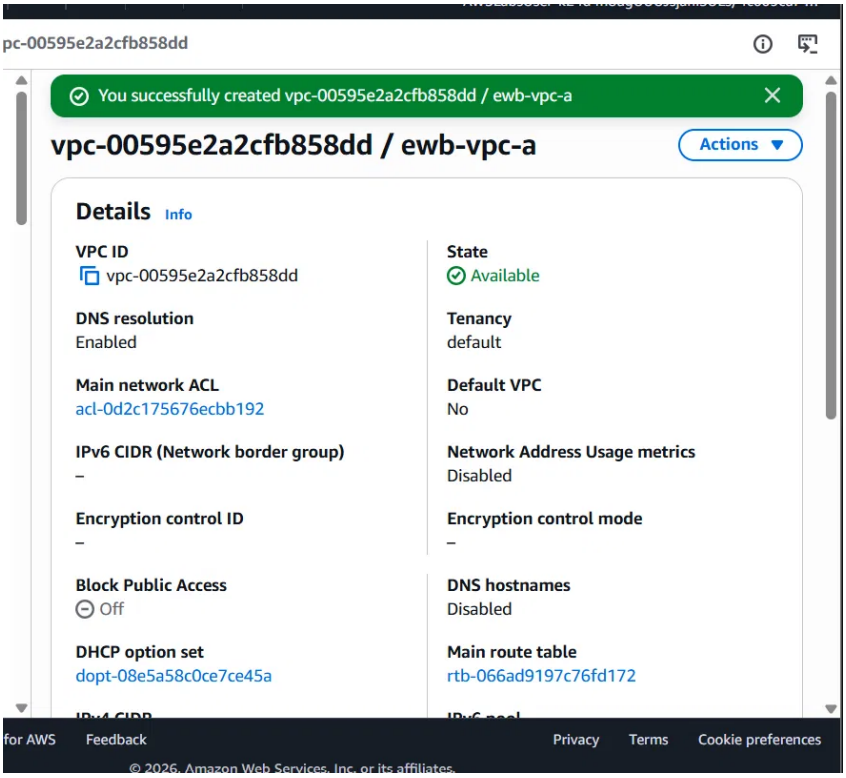


Figure 1. VPC A (ewb-vpc-a, vpc-00595e2a2cfb858dd) created and in the Available state.

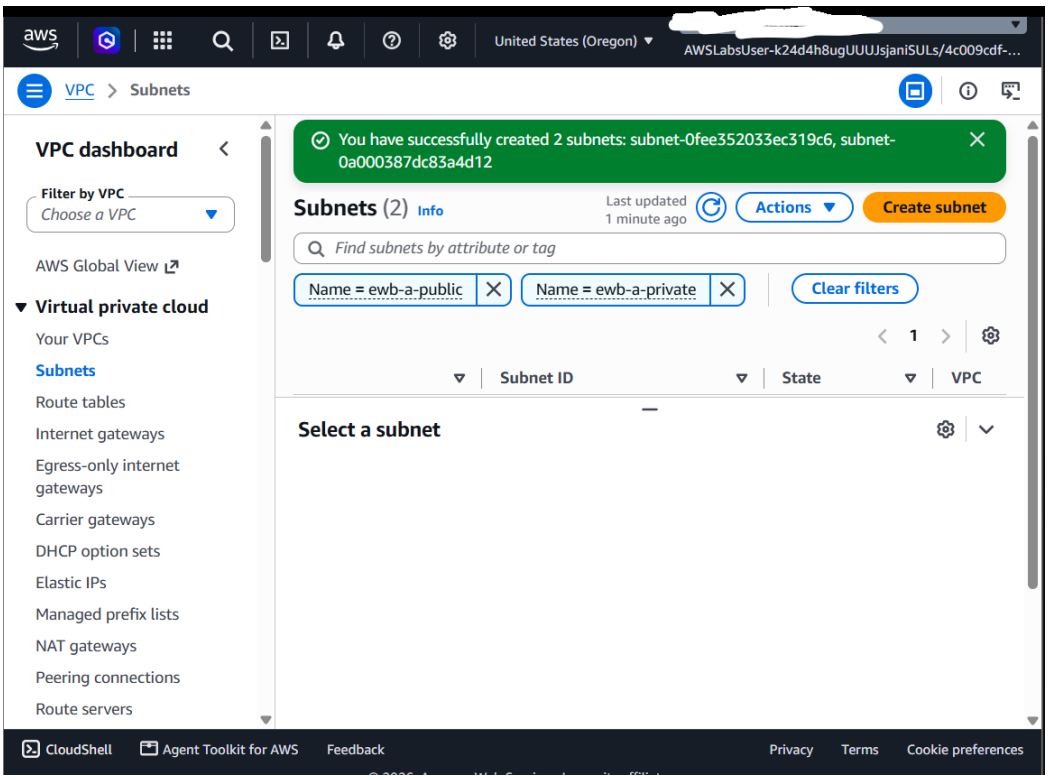


Figure 2. Public (ewb-a-public) and private (ewb-a-private) subnets created in VPC A.

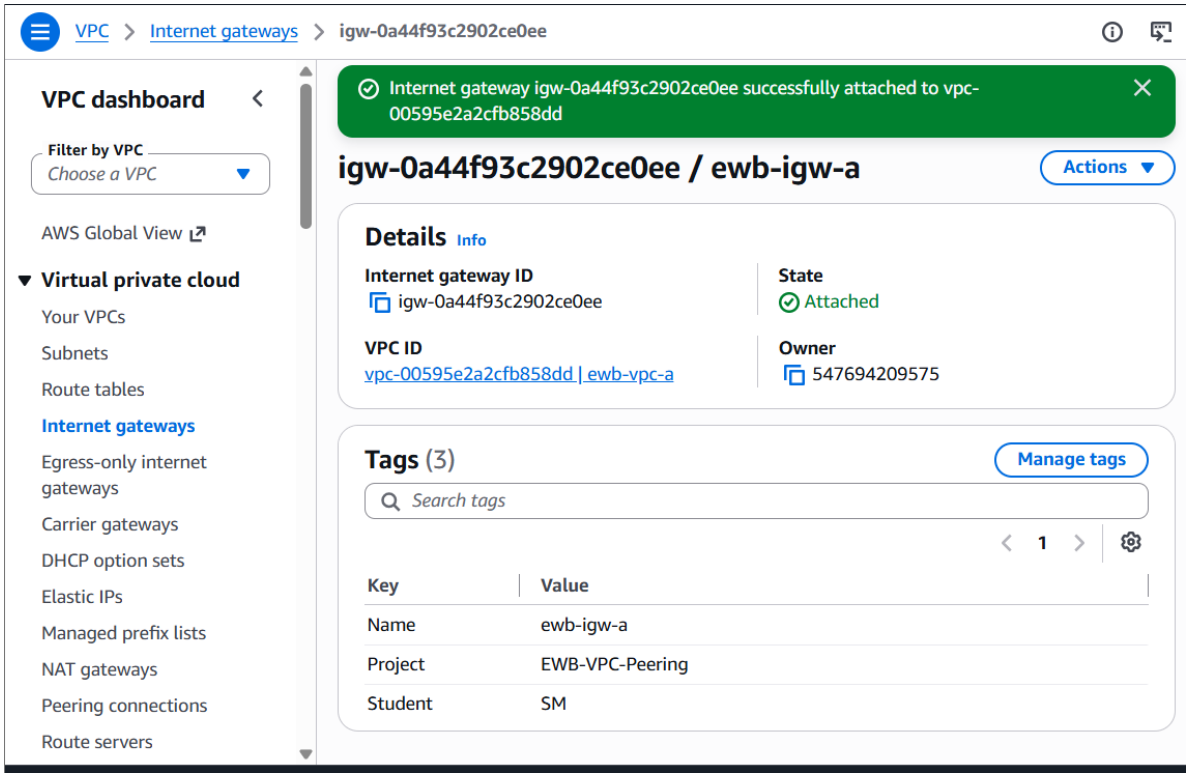


Figure 3. Internet gateway ewb-igw-a (igw-0a44f93c2902ce0ee) attached to ewb-vpc-a.

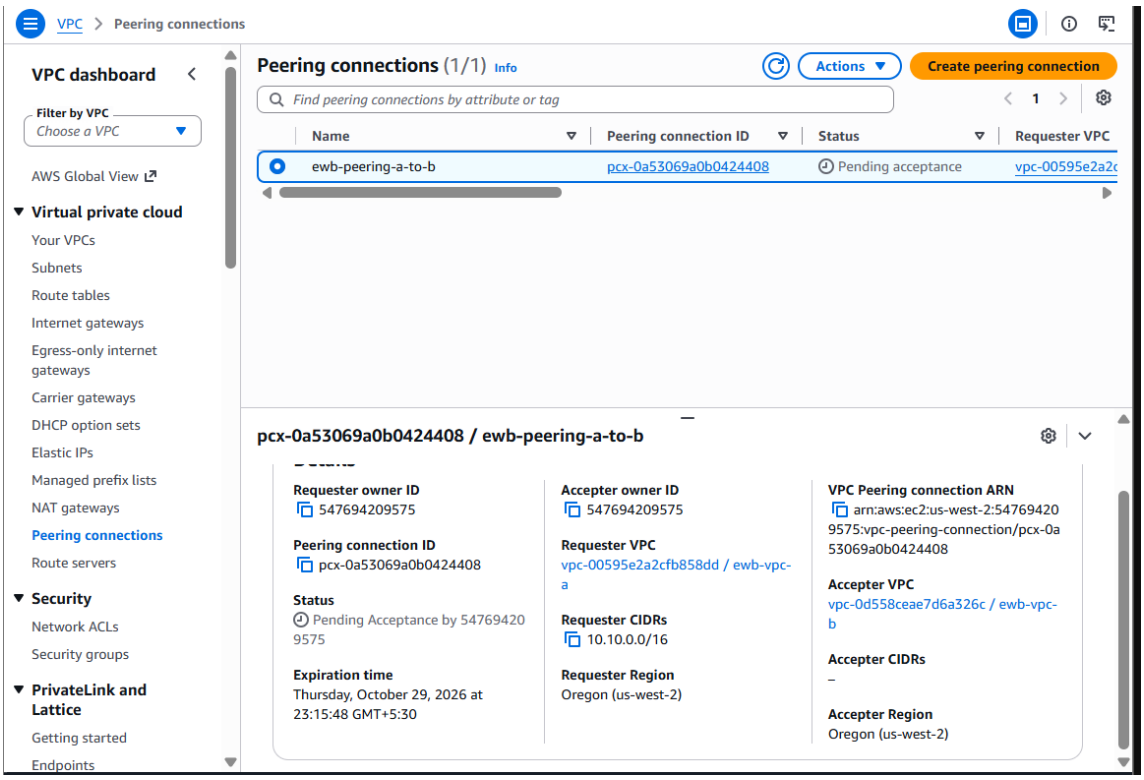


*Figure 4. Peering connection ewb-peering-a-to-b (pcx-0a53069a0b0424408) showing **Pending acceptance**.*

3. Network Routing and Security Architecture Analysis

3.1 Route table target evaluation (longest-prefix match)

A VPC route table selects the **most specific** matching destination (longest prefix). Every table contains an immutable `local` route for its own VPC CIDR. If no route matches, the packet is dropped. A route whose target is a peering connection that is not Active is shown as **Blackhole** and matching traffic is silently discarded.

Source -> destination	Table consulted	Matching route	Outcome
App-A -> App-B (10.20.1.10)	ewb-rt-a-private	10.20.0.0/16 -> pcx	Forwarded over peering (once Active)
App-B -> App-A (10.10.2.10)	ewb-rt-b-private	10.10.0.0/16 -> pcx	Forwarded over peering (once Active)
Bastion -> App-B	ewb-rt-a-public	10.20.0.0/16 -> pcx (/16 beats 0.0.0.0/0)	Forwarded over peering (requires the route)
Bastion -> Internet	ewb-rt-a-public	0.0.0.0/0 -> igw	Exits through IGW
App-A -> Internet	ewb-rt-a-private	No match	Dropped (no NAT, by design)
App-B -> Internet	ewb-rt-b-private	No match	Dropped (no IGW in VPC B)

3.2 Stateful security group mechanics

Security groups are **stateful**: when traffic is permitted in one direction, the response is automatically permitted regardless of outbound rules. App-B therefore needs only an inbound ICMP *Echo Request* rule for App-A; the Echo Reply is allowed automatically. Rules are allow-only with an implicit deny, and each source is scoped to a single host (/32) following least privilege. Security groups filter packets but do not create paths, so **routing and filtering are independent controls**, and both must permit traffic. (Network ACLs, by contrast, are stateless and need rules in both directions; the default ACL allows all.)

3.3 Bastion, ProxyJump and the public route table

App-A and App-B have no public IP addresses and cannot be reached from the internet. All administration flows through one hardened entry point, the bastion in the public subnet, whose security group accepts SSH only from the administrator's /32.

```
Laptop --SSH/22--> Bastion (10.10.1.10, public subnet, Admin /32 only)
                     |--SSH/22, private IP--------> App-A (10.10.2.10)
                     |--SSH/22, over peering------> App-B (10.20.1.10)
```

With SSH **ProxyJump** (`-J`) the laptop opens one end-to-end encrypted session whose transport hops through the bastion. The private key never leaves the laptop.

```
ssh -i ewb-key.pem -J ec2-user@<BASTION_PUBLIC_IP> ec2-user@10.20.1.10

# ~/.ssh/config equivalent
Host ewb-bastion
    HostName <BASTION_PUBLIC_IP>
    User ec2-user
    IdentityFile ~/ewb-key.pem
Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile ~/ewb-key.pem
    ProxyJump ewb-bastion
```

Why the public route table needs a route for 10.20.0.0/16. The second SSH hop is initiated by the bastion, so the route table of *its* subnet (ewb-rt-a-public) decides where the packet goes. Without `10.20.0.0/16 -> pcx-0a53069a0b0424408`, traffic to 10.20.1.10 matches only `0.0.0.0/0 -> igw` and is sent to the Internet Gateway, which cannot deliver to a private address, so the hop fails. Adding the more specific /16 peering route wins by longest-prefix

match while all other traffic still uses the IGW.

> **Configuration status**
> In the captured evidence, ewb-rt-a-public contains only the local route and the IGW default route. The `10.20.0.0/16 -> pcx-0a53069a0b0424408` entry is documented here as a **required addition** to be applied before Test 04, since it is mandatory for the bastion-to-App-B path.

3.4 Properties of VPC peering relevant to this design

- **Non-transitive:** peering gives access only to the peer VPC's own resources, never to its IGW, NAT gateway, VPN or Direct Connect.
- **Private path:** traffic stays on the AWS backbone without NAT, so original source IPs are preserved.
- **Both sides must route, and the request must be accepted** by the accepter before it becomes Active.

3.5 Screenshot evidence

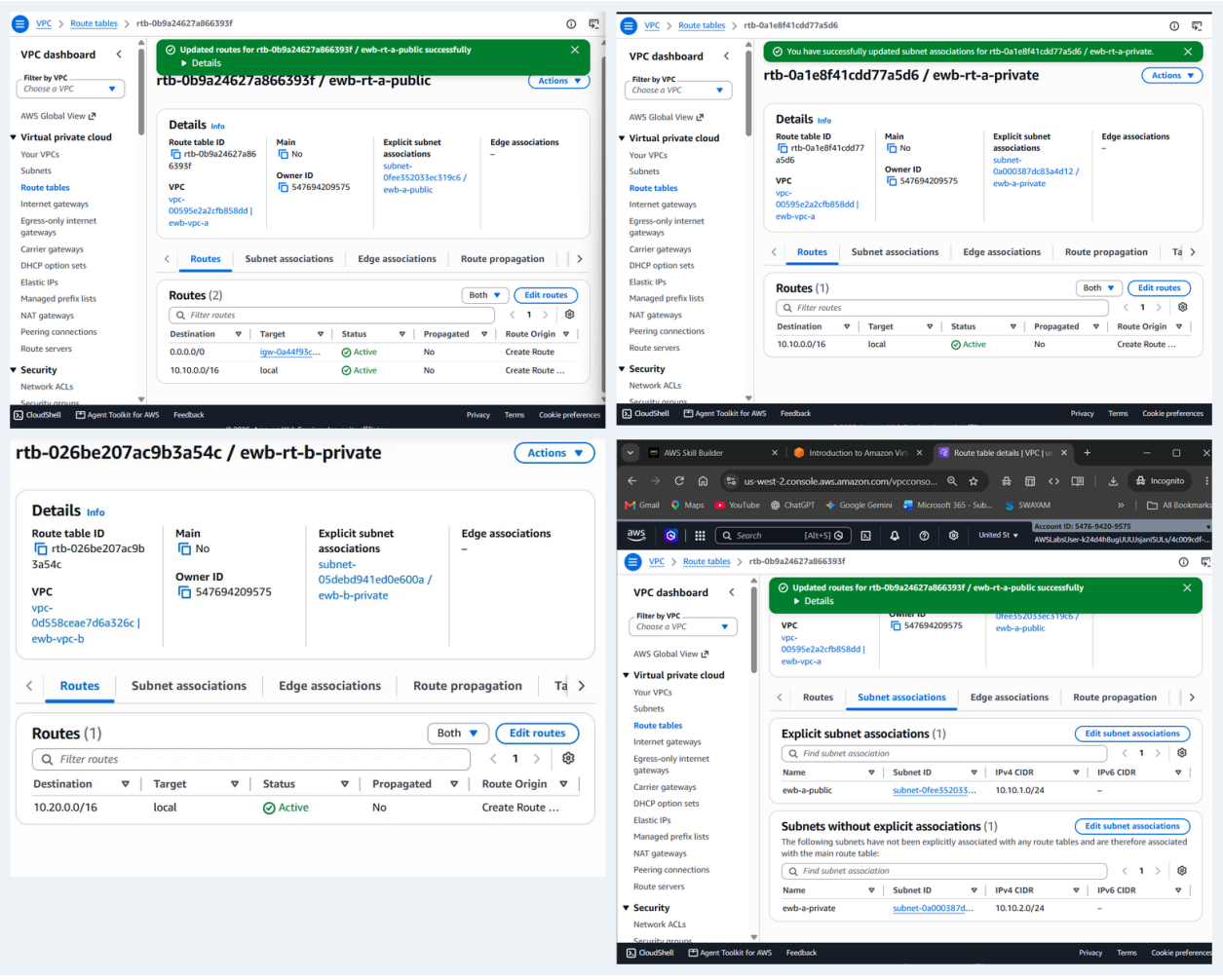


Figure 5. Route tables (top left to bottom right): ewb-rt-a-public routes, ewb-rt-a-private routes, ewb-rt-b-private routes and ewb-rt-a-public subnet associations. Peering routes show Blackhole while the connection is pending.

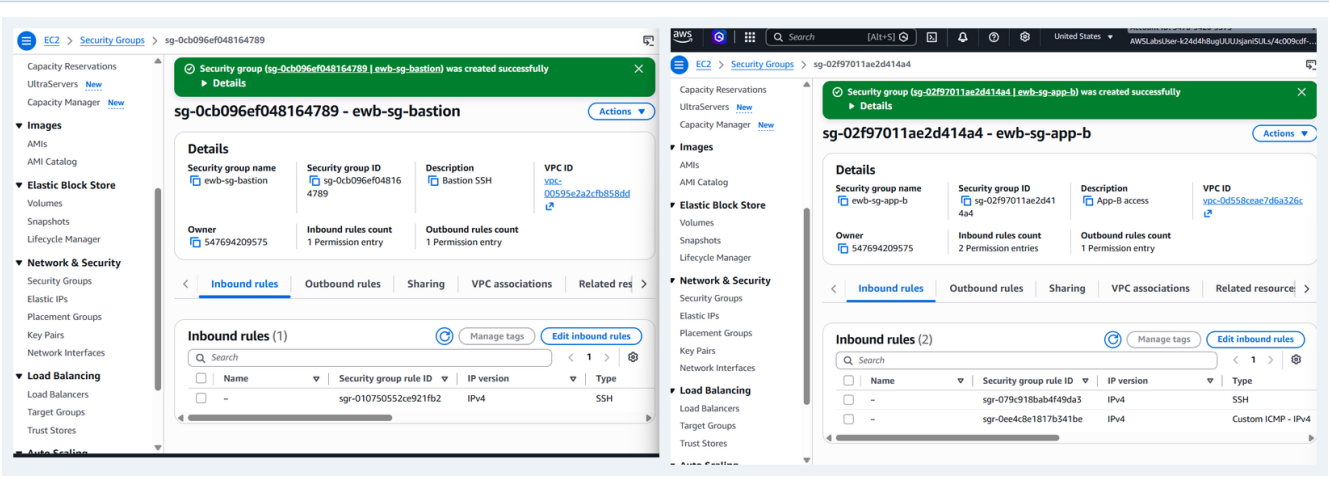


Figure 6. Inbound rules for ewb-sg-bastion (left) and ewb-sg-app-b (right). The ewb-sg-app-a screenshot was not captured.

4. Testing and Verification Mechanics (Tests 01 to 05)

Status of this section

Tests 01 to 05 are specified as a validation procedure with **expected results derived from the configured routing and security state**. They were **not executed** on live instances because the sandbox denied compute launch and peering acceptance (Section 5). No terminal output is presented as observed.

Test 01: Baseline failure (no usable peering path)

Purpose	Prove that the two VPCs are isolated by default.
Preconditions	No active peering route between the VPCs (peering absent, pending or route not yet added). App-A and App-B running.
Expected result	`4 packets transmitted, 0 received, 100% packet loss`. The route for 10.20.0.0/16 is missing or Blackhole, so packets never leave VPC A.
Conclusion	Separate VPCs cannot communicate until peering is Active and routes exist. The current Pending/Blackhole state is exactly this baseline.

Procedure

```
# On App-A
ping -c 4 10.20.1.10
```

Test 02: App-A to App-B ping across the peering connection

Purpose	Show private connectivity from VPC A to VPC B.
Preconditions	Peering Active; routes present on both sides; ewb-sg-app-b allows ICMP Echo Request from 10.10.2.10/32.
Expected result	`4 packets transmitted, 4 received, 0% packet loss`.
Conclusion	Forward path (App-A route + peering + App-B security group) is functional.

Procedure

```
# On App-A
ping -c 4 10.20.1.10
```

Test 03: App-B to App-A ping (reverse direction)

Purpose	Show that connectivity is bidirectional and the return route works.
Preconditions	As Test 02, plus ewb-sg-app-a allows ICMP Echo Request from 10.20.1.10/32.
Expected result	`4 packets transmitted, 4 received, 0% packet loss`.
Conclusion	Both route tables and both security groups are correctly configured.

Procedure

```
# On App-B
ping -c 4 10.10.2.10
```

Test 04: Private SSH via ProxyJump (Laptop -> Bastion -> App-B)

Purpose	Prove that App-B is reachable for administration only through the bastion.
Preconditions	Instances running; ewb-rt-a-public has the 10.20.0.0/16 peering route; App-B SG allows TCP 22 from 10.10.1.10/32; bastion SG allows TCP 22 from the Admin /32.
Expected result	Jump login succeeds and App-B sees the connection from 10.10.1.10. The direct attempt times out because 10.20.1.10 is not routable from the internet.
Conclusion	Single controlled entry point confirmed; private instances are not exposed.

Procedure

```
# Allowed: via bastion
ssh -i ewb-key.pem -J ec2-user@<BASTION_PUBLIC_IP> ec2-user@10.20.1.10
hostname -I            # expect 10.20.1.10

# Must fail: direct from laptop
ssh -i ewb-key.pem ec2-user@10.20.1.10
```

Test 05: Controlled ICMP failure and recovery

Purpose	Show that the security group, not the route, controls ICMP, and that the fault is reversible.
Preconditions	Test 02 passing.
Expected result	0% loss, then 100% loss after rule removal (silent drop, not 'unreachable'), then 0% loss after restore.
Conclusion	Routing and peering remained healthy throughout; only filtering changed. This separates routing faults from security group faults when troubleshooting.

Procedure

```
1) ping -c 4 10.20.1.10        # App-A: 0% loss
2) Delete the ICMP rule from ewb-sg-app-b
3) ping -c 4 10.20.1.10        # App-A: 100% loss (timeouts)
4) SSH via bastion to App-B    # still works (only ICMP removed)
5) Re-add ICMP Echo Request from 10.10.2.10/32
6) ping -c 4 10.20.1.10        # App-A: 0% loss
```

5. Sandbox Environmental Constraint Notice

This project was executed in a managed AWS Skill Builder sandbox under the IAM role **AWSLabsUser**. The role's policy **explicitly denies** two actions required to complete the end-to-end validation:

- `ec2:RunInstances`: no EC2 instance (bastion, App-A, App-B) could be launched.
- `ec2:AcceptVpcPeeringConnection`: the peering request created from VPC A could not be accepted on the VPC B side, so it remains in **Pending acceptance**.

In IAM policy evaluation an explicit deny overrides any allow, so neither action can be performed in this account regardless of other permissions. The Blackhole status on both peering routes is the expected consequence of the connection not being Active, and is consistent with the console evidence in Figures 4 and 5.

Layer	Status
VPCs, subnets, Internet Gateway	Built and verified
Route tables and subnet associations	Built and verified
Peering connection request and static routes	Built; connection **Pending acceptance** (acceptance denied by IAM); routes Blackhole
Security groups and rules	Built and verified
Peering route in ewb-rt-a-public (10.20.0.0/16)	Documented as required; not present in captured evidence
EC2 instances (Bastion, App-A, App-B)	**Not launched**: ec2:RunInstances explicitly denied
Live ping and SSH tests (01 to 05)	**Not executable** without compute and an Active peering

All network and security infrastructure that the tests depend on was configured, so validation can run as soon as the peering is accepted and compute permissions exist. Expected outcomes in Section 4 follow deterministically from longest-prefix route evaluation, stateful security group behaviour and the non-transitive nature of peering.

6. Technical Viva and Evaluation Questions

Q1. Why is a return route required in VPC B?

Peering routing is neither automatic nor symmetric. When App-A pings App-B, the Echo Reply from 10.20.1.10 is addressed to 10.10.2.10. App-B's subnet consults ewb-rt-b-private; with only `10.20.0.0/16 -> local` there is no match for 10.10.0.0/16 and the reply is dropped. Stateful security groups permit the reply *logically*, but statefulness does not create a *path*. Each VPC needs a route to the other's CIDR via the peering connection.

Q2. Why is the SSH source for App-B the bastion's private IP?

With ProxyJump the TCP connection arriving at App-B originates from the bastion instance, not the laptop. It crosses the peering connection using private addressing with no NAT, so App-B sees source 10.10.1.10. The bastion's public IP never appears on this path. Using 10.10.1.10/32 also applies least privilege: only that host can open SSH sessions to App-B, and no internet-routable address is trusted. (A production alternative is referencing the bastion's security group as the source, which survives IP changes.)

Q3. Why does Active peering state alone not guarantee ICMP reachability?

Active only means both sides agreed to the link (control plane). Data-plane reachability also requires: (1) routes in **both** subnets' route tables to the remote CIDR via the peering connection; (2) security groups that permit ICMP; (3) network ACLs that allow it in both directions (they are stateless); (4) non-overlapping CIDRs; and (5) no OS-level firewall blocking ICMP. A failure at any one layer produces silent drops, as demonstrated in Test 05.

Q4. Why can VPC B not access the Internet via VPC A's Internet Gateway?

VPC peering is **non-transitive** and does not support edge-to-edge routing. Traffic arriving over a peering connection can only be delivered to resources inside the peer VPC; it is never forwarded onward to the peer's IGW, NAT gateway, VPN or Direct

Connect. In addition, App-B has no public IP and VPC B has no IGW. Internet access for VPC B would need its own IGW with public addressing, or a NAT gateway in VPC B.

7. Resource Cleanup Plan

Delete in dependency order to avoid 'has dependencies' errors, then verify nothing tagged `Project = EWB-VPC-Peering` remains.

Order	Action
1	Terminate any EC2 instances and release Elastic IPs (none were launched in this lab).
2	Delete the peering connection pcx-0a53069a0b0424408 (or let the pending request expire on Oct 29, 2026).
3	Delete the peering routes from ewb-rt-a-public (if added), ewb-rt-a-private and ewb-rt-b-private.
4	Disassociate subnets, then delete the custom route tables.
5	Detach ewb-igw-a from ewb-vpc-a, then delete it.
6	Delete security groups ewb-sg-bastion, ewb-sg-app-a, ewb-sg-app-b (remove cross-references first).
7	Delete subnets ewb-a-public, ewb-a-private, ewb-b-private.
8	Delete ewb-vpc-a and ewb-vpc-b.
9	Confirm no VPCs, IGWs or peering connections with the project tag remain.

8. Final Conclusion

This project designed and built a two-VPC architecture with non-overlapping RFC 1918 ranges (10.10.0.0/16 and 10.20.0.0/16), an internet-facing bastion tier, private application tiers, a peering connection request, bidirectional static routes and least-privilege security groups using /32 sources. The configuration illustrates the core principles of VPC peering: isolation by default, routes required on both sides, stateful security group behaviour and non-transitivity.

End-to-end validation was prevented by explicit IAM denies in the lab sandbox on `ec2:RunInstances` and `ec2:AcceptVpcPeeringConnection`. The connection therefore remains Pending acceptance and the peering routes show Blackhole. The documented test plan gives deterministic expected results that can be executed directly once the peering is accepted, the bastion's peering route is added and compute is permitted.

> **Key takeaway**
> Peering provides a *possible* path. Route tables, security groups and network ACLs decide whether packets actually take it.